

API Documentation: Internship Progress Dashboard

1. Introduction

Overview of API

The Internship Progress Dashboard API provides a secure and scalable interface for managing user authentication, tracking internship tasks, and visualizing overall progress metrics. It acts as the core backend service for a dashboard application utilized by interns, organizers, and administrators to monitor and facilitate the remote internship program.

Purpose

The primary purpose of this documentation is to provide developers with a comprehensive reference for interacting with the API, enabling seamless communication between the client application and the backend database for **Create**, **Read**, **Update**, and **Delete** (CRUD) operations on task, user, and progress data.

Base URL

All requests should be prefixed with the following Base URL:

<http://localhost:8000/api/v1>

2. Authentication

This API utilizes **JWT (JSON Web Token)**-based authentication for securing all critical endpoints.

Authentication Flow

1. A user sends credentials to the </auth/login> endpoint.
2. Upon successful validation, the server returns a signed JWT.
3. For all subsequent requests to **protected routes**, this JWT must be included in the Request Headers.
4. The server validates the JWT's signature and checks the user's role and permissions before granting access.

Token Usage in Headers

The JWT must be passed in the `Authorization` header using the `Bearer` scheme.

Header Name	Value	Description
Authorization	<code>Bearer <Your_JWT_Token></code>	Replace <code><Your_JWT_Token></code> with the actual token received from the login endpoint.
Content-Type	<code>application/json</code>	Required for all POST/PUT requests with a body.

3. User Roles

Access to endpoints is strictly controlled by the user's assigned role, ensuring proper data segregation and security.

Role	Description	Key Permissions
Admin	Full system access. Manages all users, tasks, and system configurations.	Manage all Users, Manage all Tasks, View all Dashboard Data
Organizer	Manages a specific group of interns. Can create, update, and view tasks/progress for their assigned users.	Create/Update/Delete Tasks, View Assigned User Progress
User	The intern role. Can view their own profile, tasks, and update their own progress.	Get Profile, Update Profile, Update Own Progress

4. API Endpoints

All endpoints documented below are appended to the **Base URL**.

👤 Auth APIs

Endpoint Name	Method	URL	Description
Register	POST	/auth/register	Creates a new user account, typically used for the "User" (Intern) role sign-up.

Request Headers	Request Body (JSON)	Response (Success)	Status Codes
Authorization: <i>None</i>	<pre>{ "username": "johndoe", "email": "john@example.com", "password": "securepassword123", "securepassword123", "role": "User" }</pre>	<pre>{ "message": "User registered successfully", "userId": "uuid-12345" }</pre>	201 (Created)

Endpoint Name	Method	URL	Description
Login	POST	/auth/login	Authenticates a user and returns a JWT for subsequent requests.

Request Headers	Request Body (JSON)	Response (Success)	Status Codes
Authorization: <i>None</i>	<pre>{ "email": "john@example.com", "password": "securepassword123" }</pre>	See Example 1 in Section 5.	200 (OK)

Endpoint Name	Method	URL	Description
Logout	POST	/auth/logout	Invalidates the user's current session token on the server side (session-based invalidation).

Request Headers	Request Body (JSON)	Response (Success)	Status Codes
Authorization: Bearer	None	<code>{"message": "Logout successful"}</code>	200 (OK)

👤 User APIs

Endpoint Name	Method	URL	Description
Get Profile	GET	<code>/users/profile</code>	Retrieves the profile information for the authenticated user.

Request Headers	Query Parameters	Response (Success)	Status Codes
Authorization: Bearer	None	See Example 2 in Section 5.	200 (OK)

Endpoint Name	Method	URL	Description
Update Profile	PUT	<code>/users/profile</code>	Updates the authenticated user's profile details.

Request Headers	Request Body (JSON)	Response (Success)	Status Codes
Authorization: Bearer	<code>{"username": "John A. Doe", "phone": "123-456-7890"}</code>	<code>{"message": "Profile updated successfully", "userId": "uuid-12345"}</code>	200 (OK)

📅 Internship / Task APIs

Endpoint Name	Method	URL	Description
Create Task	POST	<code>/tasks</code>	Creates a new internship task. Required Roles: Admin, Organizer.

Request Headers	Request Body (JSON)	Response (Success)	Status Codes
Authorization: Bearer	<pre>{ "title": "Implement Login API", "description": "Develop the authentication endpoints.", "assignedTo": ["uuid-12345"], "dueDate": "", "priority": "High" }</pre>	<pre>{ "message": "Task created successfully", "taskId": "task-001" }</pre>	201 (Created)

Endpoint Name	Method	URL	Description
Get Tasks	GET	<code>/tasks</code>	Retrieves all tasks relevant to the user's role (Admin: All; Organizer: Assigned interns' tasks; User: Only own tasks).

Request Headers	Query Parameters	Response (Success)	Status Codes
Authorization: Bearer	<code>status</code> : (Optional) Filter by status (Pending, In Progress, Completed). <code>priority</code> : (Optional) Filter by priority (Low, Medium, High).	See Example 3 in Section 5.	200 (OK)

Endpoint Name	Method	URL	Description
Update Task	PUT	<code>/tasks/<taskId></code>	Modifies an existing task's details. Required Roles: Admin, Organizer.

| Request Headers | Path Parameters | Request Body (JSON) | Response (Success) |
Status Codes |
|---|---|---|---|
| Authorization: Bearer | `taskId`: The ID of the task to update. | `{ "description" :`

```
"Develop and deploy the authentication endpoints.", "priority":  
"Urgent"} | {"message": "Task updated successfully", "taskId":  
"task-001"} | 200 (OK) |
```

Endpoint Name	Method	URL	Description
Delete Task	DELETE	/tasks/<taskId>	Deletes a task from the system. Required Roles: Admin, Organizer.
Request Headers	Path Parameters	Response (Success)	Status Codes
Authorization: Bearer	taskId: The ID of the task to delete.	{"message": "Task deleted successfully"}	204 (No Content)

📊 Progress APIs

Endpoint Name	Method	URL	Description
Update Progress	POST	/progress	Submits an update on a specific task's progress. Required Role: User.
Request Headers	Request Body (JSON)	Response (Success)	Status Codes
Authorization: Bearer	{"taskId": "task-001", "progressPercentage": 75, "status": "In Progress", "notes": "Completed the basic structure."}	{"message": "Progress updated successfully"}	200 (OK)
Endpoint Name	Method	URL	Description
Get Dashboard Data	GET	/dashboard	Retrieves aggregated data for the dashboard view. The data scope changes based on the user's role.

Request Headers	Query Parameters	Response (Success)	Status Codes
Authorization: Bearer	None	See Example 4 in Section 5.	200 (OK)

🛠️ Admin APIs

Endpoint Name	Method	URL	Description
Manage Users	GET/POST/PUT/DELETE	/admin/users	Admin endpoint for viewing, creating, updating, or deleting any user in the system. Required Role: Admin.

Request Headers	Query Parameters	Path Parameters	Response (Success)	Status Codes
Authorization: Bearer	GET: <code>role</code> : (Optional) Filter by role (User, Organizer).	PUT/DELETE: <code>userId</code> : The ID of the user to manage.	GET: List of all users. PUT: User updated message. DELETE: User deleted message.	200 (OK), 204 (No Content)

Endpoint Name	Method	URL	Description
Manage Tasks	GET/DELETE	/admin/tasks	Admin endpoint for viewing all tasks regardless of assignment, and mass deletion. Required Role: Admin.

Request Headers	Query Parameters	Path Parameters	Response (Success)	Status Codes
Authorization: Bearer	GET: <code>assignedTo</code> : (Optional) Filter tasks by user ID.	DELETE: <code>taskId</code> : The ID of the task to delete.	GET: List of all tasks. DELETE: Task deleted message.	200 (OK), 204 (No Content)

📺 5. Request & Response Examples

Example 1: Successful Login Response (POST /auth/login)

```
{  
  
  "message": "Login successful",
```

```
"token": "eyJhbGciOiJIUzI1Ni...",

"user": {

  "id": "uuid-12345",

  "email": "john@example.com",

  "role": "User"

}

}
```

Example 2: Successful Get Profile Response (GET /users/profile)

```
{

  "id": "uuid-12345",

  "username": "John A. Doe",

  "email": "john@example.com",

  "role": "User",

  "joinedDate": "<span type='placeholder' placeholder-type='date'></span>",

  "lastLogin": "<span type='placeholder' placeholder-type='date'></span>",

  "assignedOrganizer": "<span type='placeholder' placeholder-type='person'></span>",

  "status": "Active"

}
```

Example 3: Successful Get Tasks Response (GET /tasks)

```
{

  "totalTasks": 2,

  "tasks": [
```



```
{  
  
  "taskId": "task-001",  
  
  "title": "Implement Login API",  
  
  "status": "In Progress",  
  
  "priority": "High",  
  
  "progressPercentage": 75,  
  
  "dueDate": "<span type='placeholder' placeholder-type='date'></span>",  
  
  "assignedTo": ["uuid-12345"]  
  
},  
  
{  
  
  "taskId": "task-002",  
  
  "title": "Design Database Schema",  
  
  "status": "Completed",  
  
  "priority": "Medium",  
  
  "progressPercentage": 100,  
  
  "dueDate": "<span type='placeholder' placeholder-type='date'></span>",  
  
  "assignedTo": ["uuid-12345"]  
  
}  
  
]  
  
}
```

Example 4: Successful Get Dashboard Data Response (GET /dashboard)

```
{
```

```
"summary": {  
  
  "tasksCompleted": 5,  
  
  "tasksPending": 3,  
  
  "completionRate": "62.5% ",  
  
  "tasksLate": 1,  
  
  "totalUsers": 12  
  
},  
  
"progressChartData": [  
  
  {"month": "Apr", "completed": 2, "pending": 1},  
  
  {"month": "May", "completed": 3, "pending": 2}  
  
]  
  
}
```

6. Error Handling

All error responses adhere to a standard JSON structure, facilitating consistent and predictable handling by client applications.

Standard Error Response Format

```
{  
  
  "status": 401,  
  
  "error": "Unauthorized",  
  
  "message": "Access token is missing or invalid."  
  
}
```

Common Errors and Status Codes

Status Code	Error Message	Description
400 Bad Request	Invalid input format or Missing required fields	The request body or parameters were malformed, failed validation, or were incomplete.
401 Unauthorized	Missing or invalid token	The request requires authentication, but the JWT is missing, expired, or malformed.
403 Forbidden	Insufficient permissions	The user is authenticated but lacks the necessary role (e.g., a "User" trying to access an Admin API).
404 Not Found	Resource not found	The requested resource (e.g., task ID, user ID) does not exist on the server.
429 Too Many Requests	Rate limit exceeded	The user has exceeded the allowed number of requests in a given time window.
500 Internal Server Error	An unexpected error occurred	A server-side issue (e.g., database connection failure or unhandled exception).

7. Security Best Practices

Security is implemented at multiple layers to protect data integrity and user privacy.

Security Feature	Implementation Detail
Token Expiry	JWTs are short-lived (e.g., 1 hour) to mitigate unauthorized access from stolen tokens. Refresh tokens are used for long-term sessions.
Password Hashing	All passwords must be stored using strong, one-way hashing algorithms like bcrypt or Argon2 with a high-cost factor.

Security Feature	Implementation Detail
Rate Limiting	Implement rate limiting on sensitive and resource-intensive endpoints (e.g., <code>/auth/login</code> , <code>/auth/register</code>) to prevent brute-force attacks and DDoS.
Input Validation	Strict validation and sanitization of all request body and query parameters to prevent common vulnerabilities like SQL injection and XSS attacks.
HTTPS	The API must always be served over HTTPS in a production environment to ensure all data transmitted is encrypted (TLS/SSL).
Role-Based Access Control (RBAC)	All protected routes strictly enforce RBAC as outlined in Section 3, ensuring users can only access endpoints and data authorized by their role.

8. Testing APIs

We strongly recommend using industry-standard tools like **Postman** or **Swagger UI** for testing and integrating with this API.

How to Test with Postman

1. **Environment Setup:** Create a Postman environment with variables `BaseURL` set to `http://localhost:8000/api/v1` and `Token` (initially empty).
2. **Login Request:** Execute the `POST /auth/login` endpoint.
3. **Automatic Token Capture:** Use a Postman "Tests" script to automatically extract the returned token from the successful response and set the `Token` environment variable.
4. **Protected Route Usage:** For any protected route (e.g., `GET /users/profile`):
 - Set the Request Header `Authorization` to `Bearer {{Token}}`.
 - Postman will automatically substitute the current JWT, allowing access to the protected resource.

Postman Collection

The Postman collection for this API can be downloaded from: [📄 File](#)